VŠB TECHNICKÁ UNIVERZITA OSTRAVA | FAKULTA ELEKTROTECHNIKY A INFORMATIKY

Absolvování individuální odborné praxe

Individual Professional Practice in the Company

Michal Křižák

Bakalářská práce

Vedoucí práce: Ing. Gaura Jan, Ph.D.

Ostrava, 2026

Č.j.VSB/25/137510
vsboee2504056B

Zadání bakalářské práce

Student: **Michal Křižák**

Studijní program: B0613A140014 Informatika

Téma: Absolvování individuální odborné praxe
Individual Professional Practice in the Company

Jazyk vypracování: čeština

Zásady pro vypracování:

Student vykoná individuální praxi ve firmě: AVE Soft s.r.o.

Struktura závěrečné zprávy:
1. Popis odborného zaměření firmy, u které student vykonal odbornou praxi a popis pracovního zařazení studenta.
2. Seznam úkolů zadaných studentovi v průběhu odborné praxe s vyjádřením jejich časové náročnosti.
3. Zvolený postup řešení zadaných úkolů.
4. Teoretické a praktické znalosti a dovednosti získané v průběhu studia uplatněné studentem v průběhu odborné praxe.
5. Znalosti či dovednosti scházející studentovi v průběhu odborné praxe.
6. Dosažené výsledky v průběhu odborné praxe a její celkové zhodnocení.

Seznam doporučené odborné literatury:

Podle pokynů konzultanta, který vede odbornou praxi studenta.

Formální náležitosti a rozsah bakalářské práce stanoví pokyny pro vypracování zveřejněné na webových stránkách fakulty.

Vedoucí bakalářské práce: **Ing. Jan Gaura, Ph.D.**

Konzultant bakalářské práce: Ing. Martin Prokeš

Datum zadání: 01.09.2025

Datum odevzdání: 30.04.2026

Garant studijního programu: Ing. Tomáš Fabián, Ph.D.

V IS EDISON zadáno: 01.10.2025 16:58:20

Abstrakt

Tato práce shrnuje průběh individuální odborné praxe vykonané ve společnosti AVE soft s.r.o. Hlavní náplní praxe byl vývoj a modernizace modulu Evolio – Importér, který slouží pro hromadný import dat do informačního systému. Cílem bylo nahradit stávající desktopové řešení (WPF) novou webovou Single Page Aplikací (SPA).

Práce se zaměřuje na implementaci uživatelského rozhraní v technologii Angular, které umožňuje uživatelům definovat šablony importů a mapovat sloupce z Excel souborů. Dále popisuje vývoj backendové logiky ve formě C# doplňku (Add-in) pro samotné zpracování dat a spolupráci na návrhu databázové struktury. Součástí praxe byla také tvorba automatizovaných testů a řešení dalších úloh v rámci firemního vývojového cyklu.

Klíčová slova

Odborná praxe; AVE soft; IS Evolio; Angular; TypeScript; C#; SQL; automatizované testy; migrace software; webová aplikace

Abstract

This report summarizes the individual professional practice performed at AVE soft s.r.o. company. The main focus of the practice was the development and modernization of the "Evolio – Importer" module, designed for bulk data import into the information system. The goal was to replace the existing desktop solution (WPF) with a new web-based Single Page Application (SPA).

The thesis focuses on the implementation of the user interface using Angular technology, enabling users to define import templates and map columns from Excel files. It further describes the development of backend logic in the form of a C# Add-in for data processing and collaboration on the database structure design. The practice also included creating automated tests and solving other tasks within the company's development cycle.

Keywords

Professional practice; AVE soft; IS Evolio; Angular; TypeScript; C#; SQL; automated tests; software migration; web application

Poděkování

Rád bych na tomto místě poděkoval vedoucímu práce, panu Ing. Janu Gaurovi, Ph.D., za odborné vedení a cenné rady. Velké poděkování patří také společnosti AVE soft s.r.o. za možnost absolvovat odbornou praxi, a především mým kolegům z vývojového týmu za vstřícný přístup, trpělivost a odbornou pomoc při řešení zadaných úkolů.

Obsah

Seznam použitých symbolů a zkratek **7**

Seznam obrázků **8**

Seznam tabulek **9**

Úvod **11**

1 Představení společnosti a pracovní pozice **12**
1.1 Profil společnosti AVE Soft s.r.o. 12
1.2 Pracovní zařazení a role v týmu 12

2 Zadání a harmonogram praxe **14**
2.1 Specifikace zadaných úkolů 14
2.2 Analýza požadavků modulu Importér 15
2.3 Časový plán a náročnost 15

3 Realizace projektu Importér **18**
3.1 Analýza a návrh databáze 18
3.2 Serverová část a aplikační logika (Add-in) 20
3.3 Uživatelské rozhraní (Frontend) 23

4 Vývoj AI rozhraní a správa frontendových požadavků **30**
4.1 Implementace AI asistenta v prostředí AngularJS 30
4.2 Řešení chyb v uživatelském rozhraní (opravy chyb) 36

5 Automatizované testování **38**
5.1 Metodika a zařazení v testovací pyramidě 38
5.2 Strategie a nástroje 39
5.3 Implementace testovacích scénářů 40
5.4 Integrace do CI/CD (Azure DevOps) 41

5.5 Přínos testovací sady 42

6 Vazba praxe na studium **43**

6.1 Uplatnění teoretických znalostí 43

6.2 Přechod od teorie k praxi 44

7 Rozvoj kompetencí a chybějící znalosti **45**

7.1 Nově osvojené technologie 45

7.2 Chybějící procesní znalosti 46

8 Zhodnocení výsledků a přínos praxe **47**

8.1 Modul Importér 47

8.2 AI rozhraní 48

8.3 Automatizované testy 48

8.4 Kritická sebereflexe 48

8.5 Budoucí rozvoj 48

9 Závěr **49**

Literatura **50**

Přílohy **51**

Seznam použitých zkratek a symbolů

IS	–	Informační systém
WPF	–	Windows Presentation Foundation
SPA	–	Single Page Application
FE	–	Frontend (prezentační vrstva aplikace)
BE	–	Backend (logická/datová vrstva aplikace)
API	–	Application Programming Interface
REST	–	Representational State Transfer
SQL	–	Structured Query Language
UI	–	User Interface (uživatelské rozhraní)
HTML	–	HyperText Markup Language
CSS	–	Cascading Style Sheets
GIT	–	Distribuovaný systém správy verzí
AI	–	Artificial Intelligence (Umělá inteligence)
LLM	–	Large Language Model (Velký jazykový model)
UX	–	User Experience (Uživatelský prožitek)
CI/CD	–	Continuous Integration / Continuous Deployment (Průběžná integrace a nasazování)
JSON	–	JavaScript Object Notation
HTTP	–	Hypertext Transfer Protocol
CRUD	–	Create, Read, Update, Delete (Základní databázové operace)
RxJS	–	Reactive Extensions for JavaScript
POM	–	Page Object Model
ERD	–	Entity-Relationship Diagram

Seznam obrázků

3.1 Zjednodušený ER diagram databázového schématu Importer 19
3.2 Ukázka rozhraní původního desktopového modulu (neintuitivní a složité). V rozbaleném dropdownu lze vidět entity, které jsou pro běžného uživatele nicneříkající. . . 23
3.3 Dashboard nového modulu Importér s náhledem dokumentu a výběrem šablony . . . 24
3.4 Sekvenční diagram procesu zpracování importu po jednotlivých řádcích 25
3.5 Nové rozhraní pro interaktivní konfiguraci mapování sloupců 28
3.6 Ukázka úspěšného importu 29

4.1 Výchozí rozhraní AI asistenta s nabídkou předdefinovaných šablon (*prompt templates*) a volbou jazykového modelu 31
4.2 Ukázka probíhající konverzace s formátováním textu (Markdown) a analýzou nahraného dokumentu 35
4.3 Ukázka zadaného požadavku (issue) v systému GitLab popisující nekonzistentní chování tabulky při kombinaci Fixed + Grouped 36

5.1 Ukázka úspěšného běhu automatizovaných testů v release pipeline Azure DevOps . . 42

Seznam tabulek

2.1 Funkční požadavky modulu Importér 16
2.2 Nefunkční požadavky modulu Importér 17

3.1 Srovnání fází PrepareImport a DirectImport 21
3.2 Kompletní přehled komponent modulu Importér 24

4.1 Vrstvy AI modulu a jejich zodpovědnosti 32

5.1 Vrstvy testovací pyramidy a jejich pokrytí v projektu Evolio 38
5.2 Porovnání Page Object Model a Screenplay Pattern 39

Seznam použitých kódů

3.1 Implementace metody pro provedení ostrého importu 21
3.2 Uložená procedura pro aktualizaci stavu položek 22
3.3 Využití signálů pro správu stavu modulu 26
3.4 Paralelní inicializace dat pomocí forkJoin 26
3.5 Definice komponenty s OnPush strategií 26
4.1 Zpracování streamované odpovědi v AngularJS 32
4.2 Konfigurace Markdown parseru (marked.js) 34
4.3 Úprava konfigurace sloupců pro detekci konfliktu Fixed/Grouped 37
5.1 Ukázka testu vytvoření subjektu pomocí Screenplay patternu 40

Úvod

Tato práce dokumentuje průběh individuální odborné praxe, kterou jsem absolvoval ve společnosti AVE soft s.r.o. v rámci závěrečné fáze bakalářského studia na Fakultě elektrotechniky a informatiky Vysoké školy báňské – Technické univerzity Ostrava. Volba absolvovat odbornou praxi namísto klasické bakalářské práce vycházela z mého zájmu ověřit si teoretické znalosti v reálném prostředí softwarové firmy a získat zkušenost s vývojem produkčního kódu v rámci rozsáhlého informačního systému.

Motivace a cíle praxe

Hlavním technickým cílem praxe byla modernizace modulu **Importér** – nástroje pro hromadný import dat do informačního systému Evolio. Úkolem bylo navrhnout a implementovat nové webové rozhraní (Single Page Application) v technologii Angular, přispět k vývoji backendové vrstvy v podobě C# doplňku (Add-in) a podílet se na návrhu databázového schématu.

Vedle hlavního projektu byly součástí praxe dva další významné okruhy úkolů: implementace uživatelského rozhraní pro AI asistenta s podporou streamované komunikace a analýzy nahraných dokumentů, a tvorba automatizovaných UI testů v jazyce C# pomocí frameworku Selenium. Společným jmenovatelem všech činností bylo zapojení do kompletního vývojového cyklu firmy, včetně code review, verzování, průběžné integrace a nasazování (CI/CD).

Struktura bakalářské práce

Práce je členěna následovně. Kapitola 1 představuje společnost AVE soft s.r.o. a moji pracovní pozici v rámci vývojového týmu. Kapitola 2 specifikuje zadané úkoly a jejich časový rozvrh. Kapitola 3 tvoří technické jádro práce – popisuje realizaci modulu Importér ve všech třech vrstvách (databázové, aplikační a prezentační). Kapitola 4 se věnuje vývoji AI asistenta a řešení vybraných chyb ve stávajícím frontendu. Kapitola 5 dokumentuje metodiku a implementaci automatizovaných testů. Kapitoly 6 a 7 reflektují vazbu praxe na studium a shrnují nově osvojené znalosti i zjištěné nedostatky ve vzdělání. Kapitola 8 hodnotí dosažené výsledky a Závěr shrnuje přínos praxe.

Kapitola 1

Představení společnosti a pracovní pozice

Tato kapitola představuje společnost AVE Soft s.r.o., ve které probíhala odborná praxe, a popisuje pracovní zařazení v rámci vývojového týmu.

1.1 Profil společnosti AVE Soft s.r.o.

Společnost AVE Soft s.r.o. je česká softwarová firma se sídlem v Ostravě, která působí na trhu informačních technologií již od roku 1997. Firma se specializuje na vývoj komplexních informačních systémů, přičemž její vlajkovou lodí je systém **Evolio**, určený pro advokátní kanceláře a správu pohledávek. Kromě toho se společnost věnuje vývoji zakázkového softwaru, mobilních aplikací a vlastních inovativních produktů (např. Taxlio pro daňové poradce).

Sídlo společnosti se nachází v technologickém areálu v Ostravě-Pustkovci, což umožňuje úzkou spolupráci s akademickou sférou, zejména s VŠB – Technickou univerzitou Ostrava. Firma klade důraz na využití moderních technologií, jako jsou cloudové služby Microsoft Azure, platforma .NET [1] a frameworky pro vývoj webových aplikací (Angular).

1.2 Pracovní zařazení a role v týmu

V rámci odborné praxe jsem byl zařazen do vývojového týmu jako programátor. Moje práce probíhala pod vedením zkušeného konzultanta (mentora) z firmy. Hlavním cílem mého působení byla práce na projektu **Importér**, který je součástí širšího ekosystému Evolio.

Náplň práce

Moje činnost byla rozdělena do několika klíčových oblastí:

- **Vývoj frontendu (FE):** Práce na uživatelském rozhraní v technologii **Angular** [2] a jazyce **TypeScript** [3]. Hlavním úkolem bylo vytvoření moderní Single Page Application (SPA), která nahrazuje starší desktopové řešení.
- **Vývoj backendu (BE):** Implementace doplňku (Add-in) pro Importér v jazyce **C#** [4], který zajišťuje logiku zpracování dat a komunikaci s databází.
- **Návrh databáze:** Spolupráce na analýze a návrhu struktur nových databázových tabulek potřebných pro fungování importního modulu.
- **Automatizované testování:** Tvorba automatizovaných testů v jazyce C# pro zajištění kvality a stability vyvíjeného řešení.
- **Řešení FE issues:** Údržba a rozvoj existujících částí aplikace mimo hlavní projekt. Významným úkolem v této oblasti byla implementace uživatelského rozhraní pro AI asistenta (chat), která zahrnovala výběr jazykových modelů a práci s historií konverzace.

Jako člen týmu jsem se účastnil pravidelných porad a využíval firemní nástroje pro správu úkolů a verzování kódu.

Kapitola 2

Zadání a harmonogram praxe

Tato kapitola detailně specifikuje zadané úkoly, které byly předmětem odborné praxe, a popisuje časový harmonogram jejich plnění. Práce byla rozdělena do tří hlavních oblastí: vývoj nového modulu Importér, údržba a rozvoj stávajícího frontendu (včetně implementace AI) a tvorba automatizovaných testů.

2.1 Specifikace zadaných úkolů

2.1.1 Projekt Importér

Hlavním úkolem praxe byl vývoj nového webového rozhraní pro import dat do systému Evolio jako náhrady stávající desktopové aplikace (motivace a srovnání s původním řešením jsou podrobně rozebrány v kapitole 3). Jednalo se o vývoj zcela nové komponenty bez návaznosti na existující frontendový kód.

Klíčové požadavky:

- **Intuitivní UX/UI:** Důraz na jednoduchost, aby uživatelé mohli sami importovat entity jako advokátní a realitní spisy, pohledávky, subjekty, platby a finanční položky.
- **Implementace designu:** Kódování aplikace podle dodaných grafických návrhů s možností vlastních vylepšení.
- **Backend a databáze:** Spolupráce na návrhu databázových struktur a vývoj C# doplňku (Add-in) pro zpracování dat.

2.1.2 Vývoj AI rozhraní a správa frontendových požadavků

Druhou oblastí byla práce na stávajícím frontendu systému Evolio. Nejvýznamnějším úkolem v této části byla implementace uživatelského rozhraní pro AI asistenta pro klienta z právního sektoru.

Konkrétně šlo o vývoj chatovacího okna s výběrem jazykového modelu, podporou analýzy nahraných dokumentů a streamovaným zobrazením odpovědi v reálném čase. Technické detaily tohoto řešení jsou rozebrány v kapitole 4. Vedle tohoto hlavního úkolu jsem se průběžně podílel na řešení běžných chyb a drobných úprav v systému Evolio dle požadavků klientů.

2.1.3 Automatizované testování

Třetím pilířem praxe byla tvorba automatizovaných testů pokrývajících klíčové agendy systému Evolio (Podatelna, Fakturace, Správa případů a spisů, Evidence subjektů a plateb). Úkolem bylo rozšiřovat stávající sadu o nové scénáře a udržovat její stabilitu v rámci CI/CD procesu. Metodika, volba nástrojů a architektura testovacího projektu jsou popsány v kapitole 5.

2.2 Analýza požadavků modulu Importér

Pro přehlednost jsou níže uvedeny funkční a nefunkční požadavky formulované na základě konzultací s produktovým manažerem a firemními konzultanty. Tento přehled sloužil jako vstup pro návrh datového modelu i uživatelského rozhraní. Funkční požadavky popisující chování systému z pohledu uživatele shrnuje tabulka 2.1, nefunkční požadavky týkající se kvality a omezení řešení pak tabulka 2.2.

2.3 Časový plán a náročnost

Odborná praxe byla zahájena v srpnu 2025 a probíhá kontinuálně až do současnosti. Práce na jednotlivých úkolech neprobíhala striktně lineárně, ale prolínala se podle aktuálních priorit vývojového týmu.

Přibližné rozložení časové dotace:

- **60 %** – Vývoj modulu Importér (FE + Add-in).
- **20 %** – Implementace AI rozhraní a správa frontendových požadavků.
- **20 %** – Tvorba automatizovaných testů a ostatní činnosti.

Vývoj Importéru začal implementací frontendu. Práce na backendové části (C# Add-in) byla zahájena později, jakmile byla klientská část dostatečně připravena pro integraci. Následně probíhal vývoj obou částí souběžně.

Tabulka 2.1: Funkční požadavky modulu Importér

ID	**Název**	**Popis**
FR-01	Nahrání souboru	Uživatel může nahrát vstupní soubor ve formátu Excel (XLSX) nebo CSV.
FR-02	Definice šablony	Uživatel může vytvořit, upravit nebo smazat šablonu importu s uloženým mapováním sloupců.
FR-03	Mapování sloupců	Systém umožňuje přiřadit sloupce vstupního souboru na pole importovaného kontextu (např. Subjekt, Adresa).
FR-04	Hierarchie kontextů	Uživatel může definovat vnořené kontexty (Subjekt → Adresa → Kontakt) a více instancí téhož kontextu.
FR-05	Transformace hodnot	Systém podporuje definici převodních pravidel pro textové hodnoty na interní identifikátory.
FR-06	Dvoufázový import	Systém provádí nejdříve validační (přípravnou) fázi a teprve po potvrzení uživatele ostrý zápis dat.
FR-07	Real-time sledování	Uživatel je informován o průběhu zpracování jednotlivých řádků v reálném čase.
FR-08	Auditování a logy	Každý import je uložen včetně výsledku jednotlivých řádků pro pozdější dohledání chyb.

Tabulka 2.2: Nefunkční požadavky modulu Importér

ID	**Kategorie**	**Popis**
NFR-01	Výkon	Vykreslení mapovací obrazovky se stromem do 50 kontextů musí proběhnout bez zaznamenatelné latence (pod 200 ms).
NFR-02	Škálovatelnost	Systém musí zpracovat soubory obsahující řádově tisíce řádků bez blokace uživatelského rozhraní.
NFR-03	Bezpečnost	Přístup k modulu je omezen pomocí oprávnění ve formě rolí systému Evolio; data v tranzitu jsou chráněna protokolem HTTPS.
NFR-04	Konzistence dat	Souběžné zápisy jsou ošetřeny pomocí optimistického řízení souběhu (`SysRowVersion`).
NFR-05	Rozšiřitelnost	Definice nových typů importu je možná bez zásahu do zkompilovaného kódu (pouze přes databázovou konfiguraci).
NFR-06	Použitelnost	Běžný uživatel (klient) musí být schopen provést import bez asistence firemního konzultanta.
NFR-07	Auditovatelnost	Každá operace zápisu zaznamenává identifikátor autora a časovou značku.

Kapitola 3

Realizace projektu Importér

Tato kapitola popisuje technickou realizaci klíčové části odborné praxe – modulu Importér. Řešení je rozděleno do tří vrstev: databázové, aplikační (backend) a prezentační (frontend).

3.1 Analýza a návrh databáze

Databázová vrstva je implementována na platformě Microsoft SQL Server [5] pod vyhrazeným schématem `importer`. Architektura byla navržena s důrazem na konfigurovatelnost, což umožňuje definovat nové typy importů bez nutnosti zásahů do zkompilovaného kódu aplikace.

Model dat se skládá ze tří logických celků:

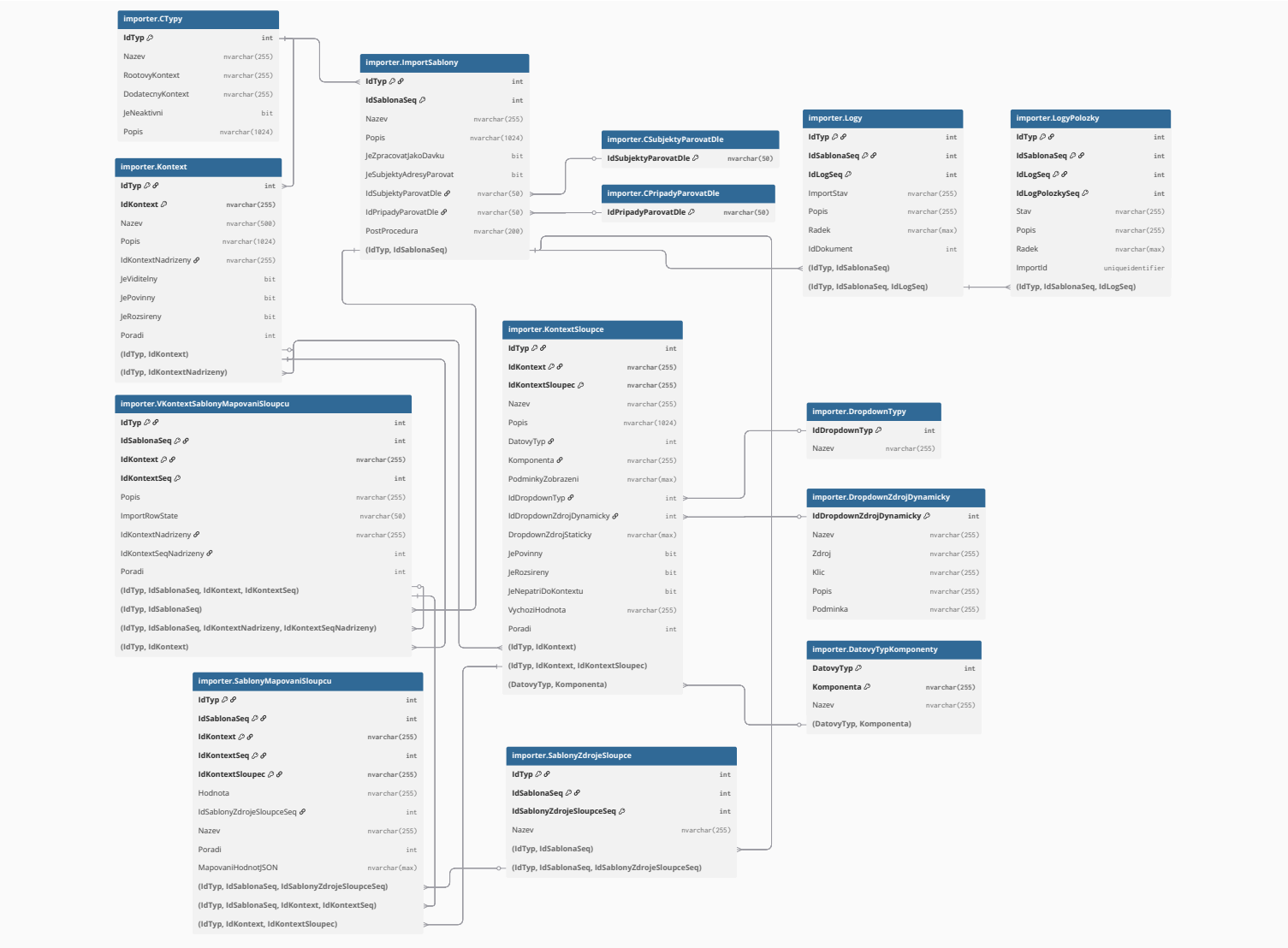
1. **Definice struktury:** Metadata popisující, jaká data lze importovat (kontexty, sloupce).
2. **Konfigurace šablon:** Uložení uživatelských nastavení a mapování.
3. **Provozní data:** Logování průběhu importu a auditní stopy.

3.1.1 Systémová infrastruktura a auditování

Všechny tabulky v rámci schématu `importer` sdílejí jednotnou sadu systémových sloupců, které zajišťují auditní stopu a integritu dat. Mezi tyto sloupce patří identifikátory autora záznamu (`SysZadal`), časové značky a sloupec `SysRowVersion` sloužící k optimistickému řízení souběhu.

3.1.2 Logické celky datového modelu

Na základě funkčnosti lze datový model rozdělit do několika klíčových oblastí, které společně zajišťují flexibilitu celého importního modulu.



Obrázek 3.1: Zjednodušený ER diagram databázového schématu Importer

3.1.2.1 Definice hierarchie a struktury

Základem pro definici toho, co lze do systému importovat, jsou tabulky `CTypy`, `Kontext` a `KontextSloupce`.

- **CTypy:** Slouží jako kořenový číselník a diskriminátor pro logickou separaci různých domén importu (např. spisy, subjekty či dokumenty).
- **Kontext:** Reprezentuje kategorii importovatelných dat — například Subjekt, Adresa nebo Kontakt. Díky rekurzivní vazbě `IdKontextNadrizeny` mohou být kontexty vzájemně vnořeny a tvoří tak stromovou strukturu (např. Subjekt → Adresa → Kontakt). Každý kontext zároveň slouží jako nadřazená kategorie pro svá konkrétní datová pole, která jsou evidována v tabulce `KontextSloupce`.
- **KontextSloupce:** Každý záznam v této tabulce představuje jedno konkrétní pole patřící do nadřazeného kontextu — například pole *Jméno* nebo *IČO* v kontextu Subjekt. Kromě datového typu obsahuje také identifikátor UI komponenty, což umožňuje dynamické vykreslování odpovídajících prvků (např. dropdown, datepicker) v prezentační vrstvě aplikace.

3.1.2.2 Konfigurace šablon a mapování

Procesní nastavení a propojení vstupních dat s vnitřní strukturou systému zajišťují entity `ImportSablony` a `SablonyMapovaniSloupcu`. Šablony uchovávají parametry jako režim dávkového zpracování nebo kritéria pro párování existujících subjektů. Tabulka `SablonyMapovaniSloupcu` pak funguje jako komplexní asociační entita, která propojuje tři nezávislé hierarchie: strukturu kontextů, zdrojové sloupce vstupního souboru a konkrétní výskyty kontextů v šabloně. Podporuje definici statických hodnot i pokročilé transformační logiky uložené v JSON formátu. Jejím hlavním úkolem je uložit stav šablony, tedy uchovat všechny její kontexty a sloupce, díky čemuž můžeme do databáze uložit kompletní informaci o hierarchii celé šablony.

3.2 Serverová část a aplikační logika (Add-in)

Serverová část modulu Importér je implementována v jazyce C# jako komponenta systému EFabric. Hlavním úkolem této vrstvy je orchestrace datového toku mezi uživatelským rozhraním, interním API systému Evolio a SQL databází. Logika je rozdělena do dvou funkčních celků (Add-inů), které zajišťují oddělení přípravné a realizační fáze importu.

3.2.1 Procesní tok: Od validace k ostrému zápisu

Zpracování importu probíhá dvoufázově. Tento přístup umožňuje uživateli zkontrolovat připravená data a odhalit případné chyby dříve, než dojde k nevratnému zápisu do produkčních tabulek systému. Porovnání parametrů a chování těchto fází uvádí tabulka 3.1.

Tabulka 3.1: Srovnání fází PrepareImport a DirectImport

Vlastnost	Fáze PrepareImport	Fáze DirectImport
Účel	Parsování souboru a příprava dat v logu.	Ostrý zápis dat do cílových entit systému.
Hlavní parametr	`PouzeUlozit = true`.	`ImportId` (získané z přípravy).
Zpracování	Hromadné pro celý soubor.	Sekvenční po jednotlivých položkách.
API Endpoint	`/api/import/v5/pripady/`.	`api/import/v5/DirectImport`.

3.2.1.1 Fáze 1: Příprava importu (PrepareImport)

V první fázi odesílá frontend metadata o souboru a zvolenou šablonu mapování. Backend volá standardní importní API systému Evolio s parametrem `PouzeUlozit = true`. Systém v tomto režimu neprovádí zápis do ostrých agend, ale parsuje vstupní Excel/CSV soubor, provádí validace a výsledná data ukládá do dočasných tabulek logu (`Logy` a `LogyPolozky`). Výsledkem tohoto volání je unikátní identifikátor `IdLogSeq` (nebo `ImportId`), který slouží jako klíč pro následné kroky.

3.2.1.2 Fáze 2: Ostrý import (DirectImport)

Jakmile uživatel potvrdí správnost připravených dat, frontend zahájí sekvenční zpracování položek. Pro každou platnou položku je vyvolána metoda `DirectImportAsync`, která využívá dříve získaný identifikátor. Tato fáze volá stejné API rozhraní, ale tentokrát pro finální uložení do databáze.

```
// Metoda zajistuje finalni zapis dat do systemu Evolio
public async Task DirectImportAsync(Guid importId)
{
    // Sestaveni URL adresy pro konkretni ImportId
    var url = string.Format("api/import/v5/DirectImport?ImportId={0}", importId);

    // Odeslani POST pozadavku na API.
    // V teto fazi dojde k fyzickemu vlozeni dat do SQL tabulek systemu.
    await Context.Post(url, string.Empty, CancellationToken.None);
}
```

Listing 3.1: Implementace metody pro provedení ostrého importu

3.2.2 Implementace databázové logiky (SQL Procedura)

Pro zajištění konzistence a výkonu při aktualizaci stavů importu využívá backend uloženou proceduru `ImporterUpdateLogyPolozkySp`. Tato procedura atomicky aktualizuje stav řádku a v případě dokončení celé dávky provádí finalizaci logu.

```sql
CREATE PROCEDURE [importer].[ImporterUpdateLogyPolozkySp]
    @SysRowid [core].[SysRowIdType], -- ID radku v tabulce LogyPolozky
    @Stav nvarchar(255),         -- S (Success) / EC (Error)
    @Popis nvarchar(255) = null  -- Detailni chybova zprava
AS
BEGIN
    SET NOCOUNT ON;
    DECLARE @IdTyp INT, @IdSablonaSeq INT, @IdLogSeq INT;

    -- 1. Aktualizace stavu konkretni polozky
    UPDATE importer.LogyPolozky
    SET Stav = @Stav,
        Popis = @Popis
    WHERE SysRowId = @SysRowid;

    -- 2. Kontrola koncovych stavu a aktualizace celkoveho logu
    IF(@Stav = 'S' OR @Stav = 'EC')
    BEGIN
        SELECT @IdTyp = IdTyp, @IdSablonaSeq = IdSablonaSeq, @IdLogSeq = IdLogSeq
        FROM importer.LogyPolozky WHERE SysRowId = @SysRowid;

        -- Overeni, zda jsou jiz vsechny polozky v davce dokoncene
        IF(NOT EXISTS(SELECT 1 FROM importer.LogyPolozky l
                    WHERE l.IdLogSeq = @IdLogSeq AND l.Stav IN ('N','R')))
        BEGIN
            -- Finalizace celeho importniho logu
            UPDATE importer.Logy SET Stav = 'D' WHERE IdLogSeq = @IdLogSeq;
        END
    END
END
```

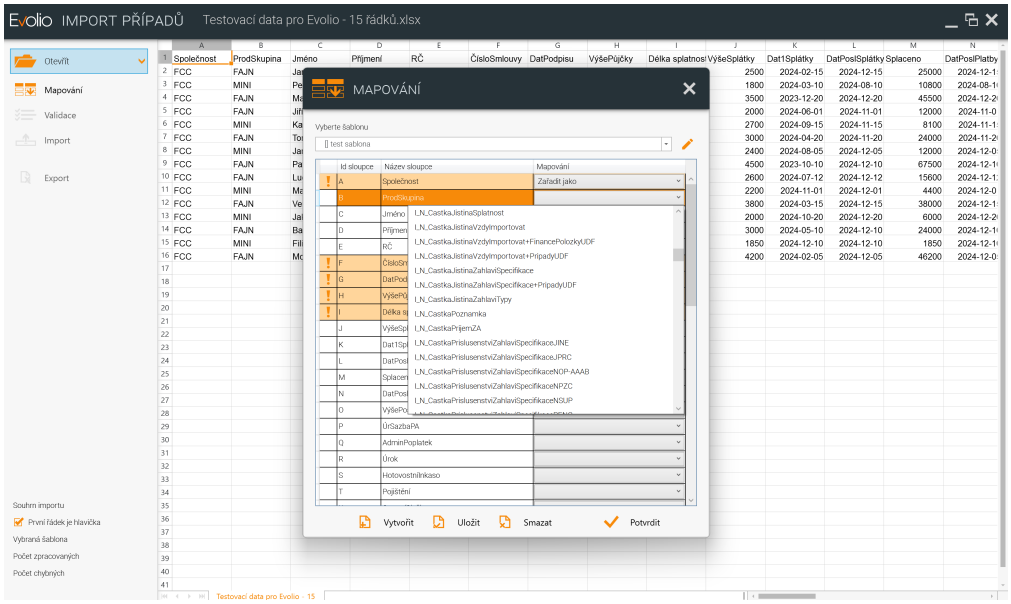
Listing 3.2: Uložená procedura pro aktualizaci stavu položek

Využití této procedury umožňuje backendu efektivně sledovat průběh importu a okamžitě reagovat na výsledky vracené z API. Pokud API vrátí chybu pro konkrétní řádek, backend zavolá proceduru se stavem `'EC'` a popisem chyby, což se ihned promítne do uživatelského rozhraní v reálném čase.

3.3 Uživatelské rozhraní (Frontend)

3.3.1 Motivace a porovnání s původním řešením

Původní modul pro import dat v systému Evolio byl realizován jako desktopová aplikace (WPF). Toto řešení bylo v průběhu let shledáno jako vysoce neintuitivní a pro běžného uživatele bez předchozího proškolení prakticky nepoužitelné (viz Obrázek 3.2). Obsluhu tohoto nástroje zvládali pouze interní firemní konzultanti.

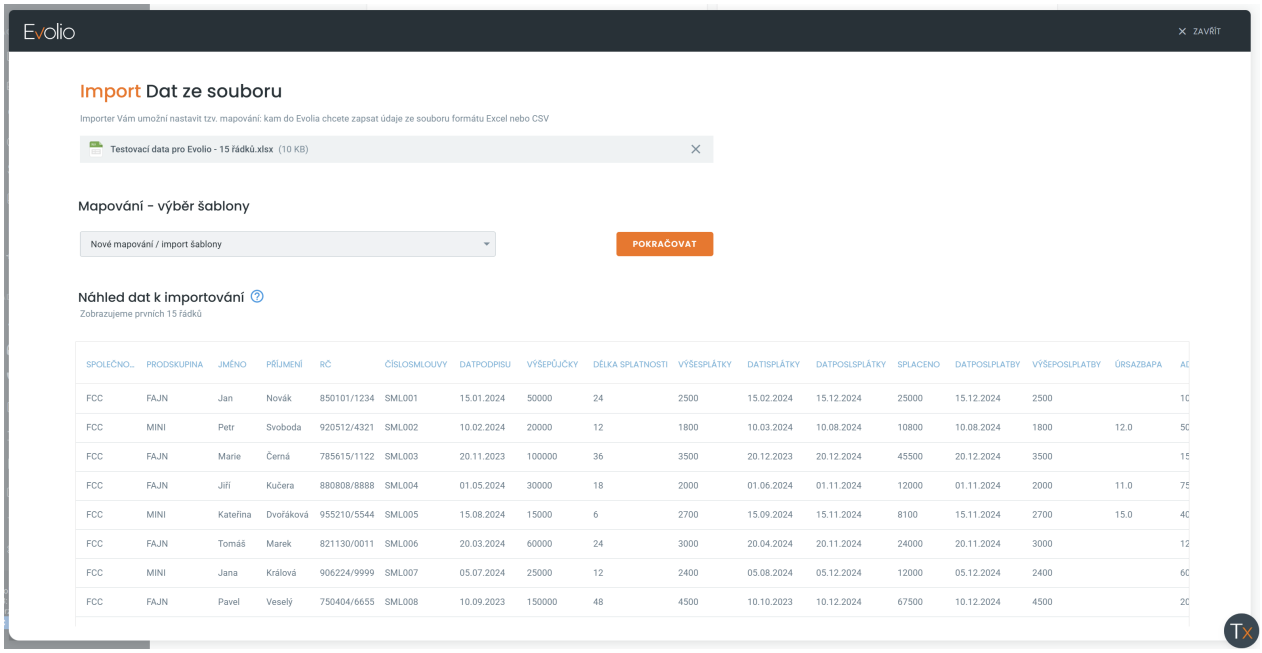


Obrázek 3.2: Ukázka rozhraní původního desktopového modulu (neintuitivní a složité). V rozbaleném dropdownu lze vidět entity, které jsou pro běžného uživatele nicneříkající.

Hlavním cílem vývoje nové webové Single Page Aplikace (SPA) bylo vytvořit moderní a intuitivní nástroj, který umožní klientům provádět importy dat zcela samostatně.

Prezentační vrstva modulu Importér představuje nejrozsáhlejší část realizace. Je navržena jako autonomní feature modul (`ImporterModule`) v rámci enterprise architektury systému Evolio, využívající verzi **Angular 17** [2] a **TypeScript** [3].

3.3.2 Architektura a hierarchie komponent



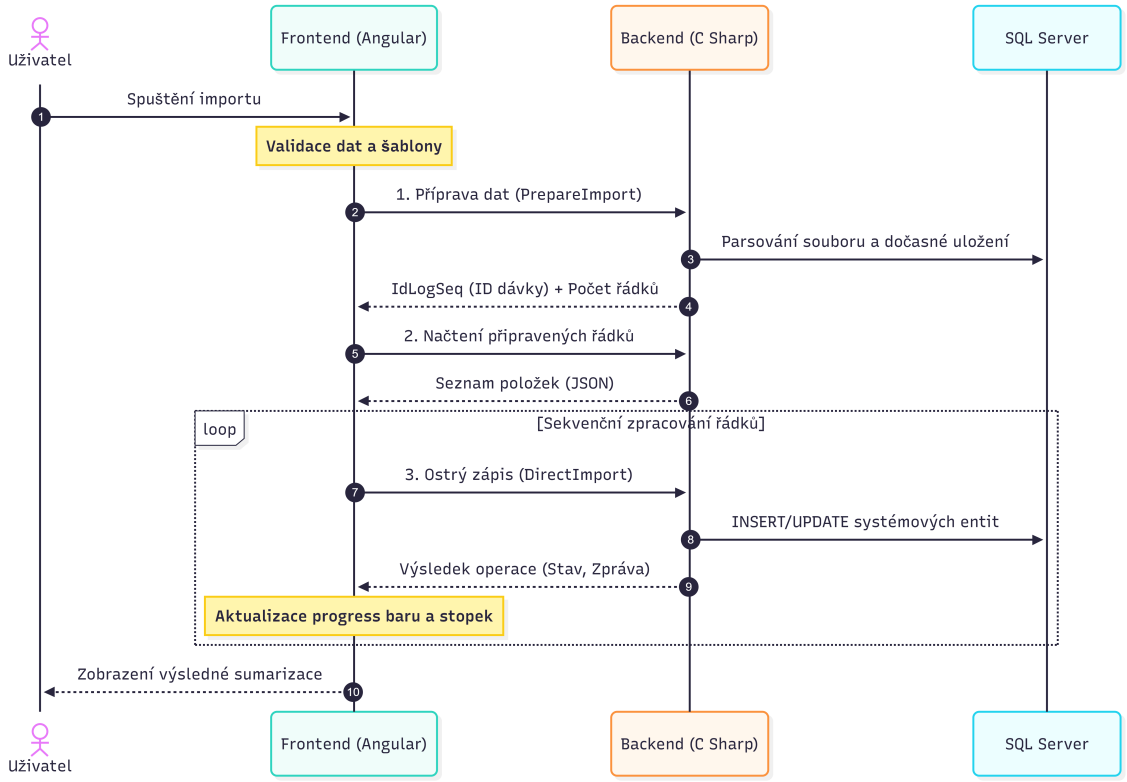
SPOLEČNO...	PRODSKUPINA	JMÉNO	PŘÍJMENÍ	RČ	ČÍSLOSMLOUVY	DATPODPISU	VÝŠEPŮJČKY	DÉLKA SPLATNOSTI	VÝŠESPLÁTKY	DATISPLÁTKY	DATPOSLSPLÁTKY	SPLACENO	DATPOSLPLATBY	VÝŠEPOSLPLATBY	ÚRSAZBAPA	AD
FCC	FAJN	Jan	Novák	850101/1234	SML001	15.01.2024	50000	24	2500	15.02.2024	15.12.2024	25000	15.12.2024	2500		10
FCC	MINI	Petr	Svoboda	920512/4321	SML002	10.02.2024	20000	12	1800	10.03.2024	10.08.2024	10800	10.08.2024	1800	12.0	50
FCC	FAJN	Marie	Černá	785615/1122	SML003	20.11.2023	100000	36	3500	20.12.2023	20.12.2024	45500	20.12.2024	3500		15
FCC	FAJN	Jiří	Kučera	880808/8888	SML004	01.05.2024	30000	18	2000	01.06.2024	01.11.2024	12000	01.11.2024	2000	11.0	75
FCC	MINI	Kateřina	Dvořáková	955210/5544	SML005	15.08.2024	15000	6	2700	15.09.2024	15.11.2024	8100	15.11.2024	2700	15.0	40
FCC	FAJN	Tomáš	Marek	821130/0011	SML006	20.03.2024	60000	24	3000	20.04.2024	20.11.2024	24000	20.11.2024	3000		12
FCC	MINI	Jana	Králová	906224/9999	SML007	05.07.2024	25000	12	2400	05.08.2024	05.12.2024	12000	05.12.2024	2400		60
FCC	FAJN	Pavel	Veselý	750404/6655	SML008	10.09.2023	150000	48	4500	10.10.2023	10.12.2024	67500	10.12.2024	4500		20

Obrázek 3.3: Dashboard nového modulu Importér s náhledem dokumentu a výběrem šablony

Modul sestává z osmi specializovaných komponent, které spolupracují na zajištění plynulého uživatelského zážitku. Přehled všech komponent uvádí tabulka 3.2.

Tabulka 3.2: Kompletní přehled komponent modulu Importér

Komponenta	**Účel a hlavní funkce**
`ImporterDetailComponent`	Hlavní dashboard modulu s historií a náhledem.
`ImporterMappingComponent`	Obrazovka pro definici mapování sloupců na kontexty.
`ContextTreeComponent`	Rekurzivní komponenta pro vizualizaci stromu kontextů.
`ImporterDropdownComponent`	Výběrový prvek pro intuitivní přiřazování sloupců.
`ValueMappingModalComponent`	Dialog pro definici transformací hodnot (převod textů na ID).
`DocumentPickerButtonComponent`	Prvek pro nahrávání souborů s podporou validace.
`ImportProgressDialogComponent`	Sleduje průběh importu po řádcích v reálném čase.
`ImportLogDetailComponent`	Detailní prohlížeč výsledků a chyb z proběhlých importů.



Obrázek 3.4: Sekvenční diagram procesu zpracování importu po jednotlivých řádcích

3.3.3 Moderní reaktivní vzory: Signály a RxJS

Aplikace využívá kombinaci tradiční knihovny RxJS [6] a nového reaktivního primitiva **Signals**, představeného v Angularu 17. Signály jsou použity pro správu synchronního stavu, který nevyžaduje komplexní transformace, což zjednodušuje kód a zvyšuje čitelnost.

V rámci `ImporterService` jsou signály využity pro sledování aktuálního kontextu:

```
// Definice signálu pro sledování vybrané šablony
public selectedSablonaId = signal<number | null>(null);

// Výpočetní signál závislý na změně ID
public isSablonaSelected = computed(() => this.selectedSablonaId() !== null);
```

Listing 3.3: Využití signálů pro správu stavu modulu

Pro asynchronní operace, jako je paralelní načítání číselníků při inicializaci, je nadále využíváno RxJS. Operátor `forkJoin` zajišťuje, že se aplikace vykreslí až v momentě, kdy jsou k dispozici všechna potřebná metadata:

```
forkJoin({
  typy: this.emap.get('cTypy', { IdTyp: this.idTyp() }),
  kontexty: this.emap.get('kontext', { IdTyp: this.idTyp() }),
  sablony: this.emap.get('importSablony', { IdTyp: this.idTyp() })
}).pipe(takeUntil(this.destroy$)).subscribe(data => {
  this.state.update(s => ({ ...s, ...data }));
});
```

Listing 3.4: Paralelní inicializace dat pomocí forkJoin

3.3.4 Optimalizace výkonu pomocí OnPush strategie

Z důvodu vysokých nároků na výkon při zobrazování rozsáhlých mapovacích struktur (stromy s desítkami uzlů) byla u všech komponent s vyššími nároky na výkon nasazena strategie **ChangeDetectionStrategy.OnPush**.

Tato optimalizace instruuje Angular, aby kontroloval změny v komponentě pouze v případě, že se změní její vstupní reference (`@Input`) nebo je změna explicitně vyžádána. Tím se výrazně redukuje počet cyklů detekce změn při interakci uživatele s formulářem.

```
@Component({
  selector: 'ev-importer-mapping',
  templateUrl: './importer-mapping.component.html',
  styleUrls: ['./importer-mapping.component.scss'],
```

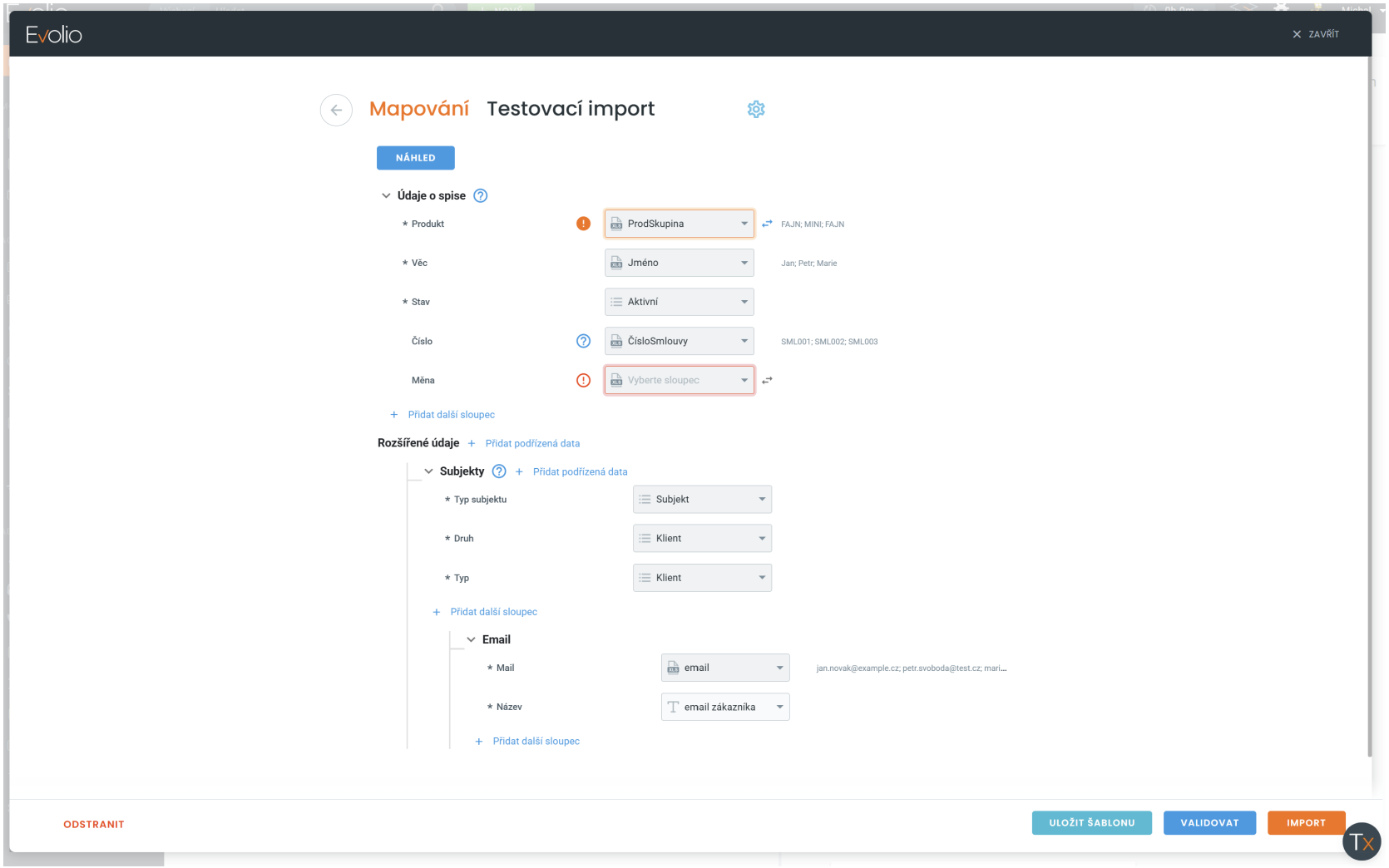
```
  changeDetection: ChangeDetectionStrategy.OnPush
})
export class ImporterMappingComponent { ... }
```

Listing 3.5: Definice komponenty s OnPush strategií

3.3.5 Implementace komplexní mapovací logiky

Jádrem modulu je `ImporterMappingComponent` (viz Obrázek 3.5), která umožňuje uživateli interaktivně sestavit strukturu importu. Na rozdíl od původního desktopového řešení se zde uplatňuje princip budování struktury na vyžádání (*on-demand*):

- **Dynamické instance:** Uživatel může přidávat více instancí stejného kontextu (např. více adres k jednomu subjektu). Každá instance má vlastní stav mapování sloupců.
- **Interaktivní správa:** Stav sbalení a rozbalení jednotlivých sekcí je udržován v `Set<string>`, což zajišťuje rychlou manipulaci i při velkém počtu prvků.
- **Datová transformace:** Pomocí *pure pipes* dochází k transformaci dat bez nutnosti přepočítávání při každém cyklu detekce změn.

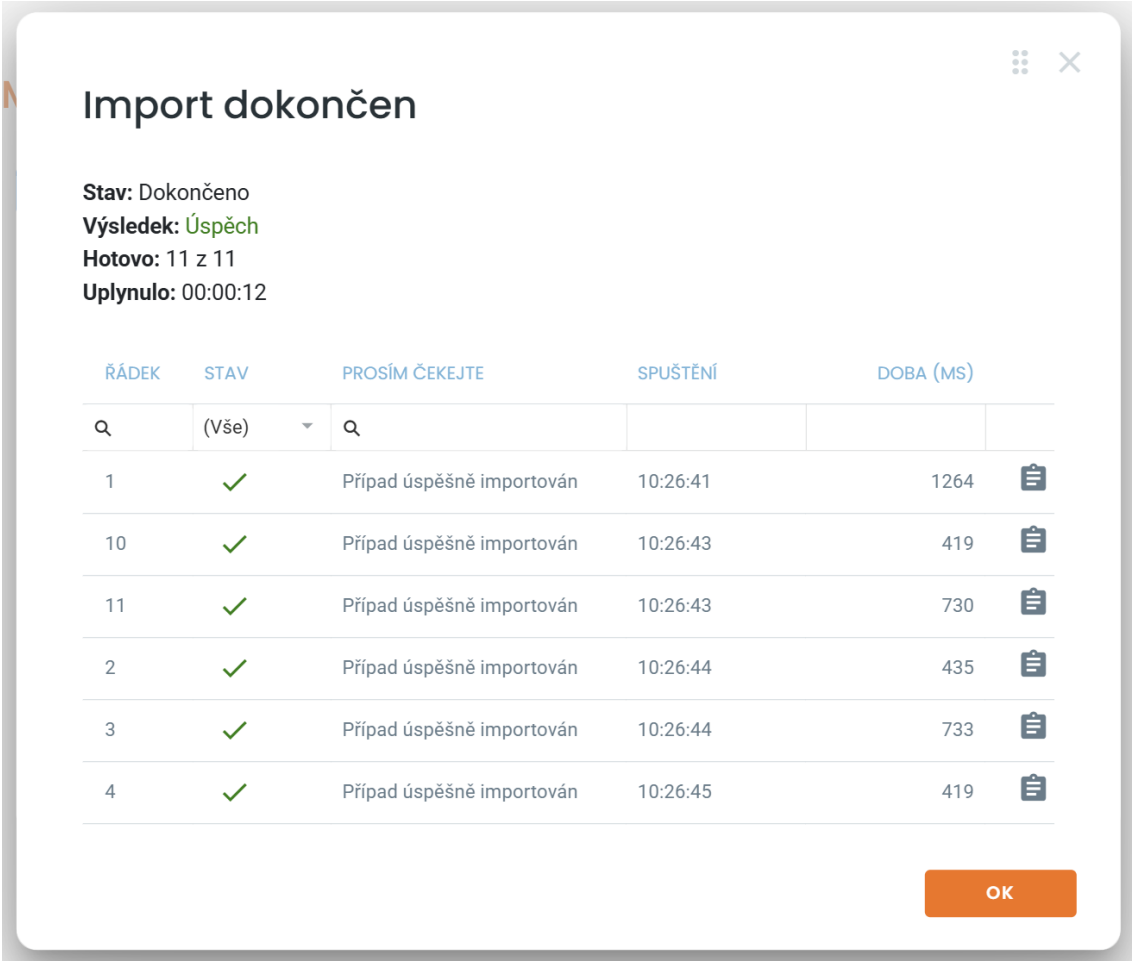


Obrázek 3.5: Nové rozhraní pro interaktivní konfiguraci mapování sloupců

3.3.6 Sledování v reálném čase a UX prvky

Proces importu probíhá sekvenčně, přičemž uživatel je o postupu informován prostřednictvím `ImportProgressDialogComponent`. Tato komponenta kombinuje několik technologií pro zajištění plynulé zpětné vazby:

- **Sledování času:** Využití knihovny *moment.js* v sekundovém intervalu pro zobrazení doby trvání importu.
- **Inkrementální DataGrid:** Výsledky jsou do tabulky (*DevExtreme DataGrid*) přidávány okamžitě po dokončení požadavku na konkrétní řádek.
- **Vizuální indikátory:** Namísto statických spinnerů jsou v dashboardu využity tzv. *skeleton loadery* – zástupné vizuální placeholdery, které predikují strukturu obsahu a snižují kognitivní zátěž uživatele při čekání na data.



Obrázek 3.6: Ukázka úspěšného importu

Kapitola 4

Vývoj AI rozhraní a správa frontendových požadavků

Vedle hlavního projektu Importér byla významná část času věnována práci na klientských požadavcích a údržbě existujících systémů. Tato kapitola popisuje vývoj specializovaného AI modulu a řešení specifických chyb v uživatelském rozhraní.

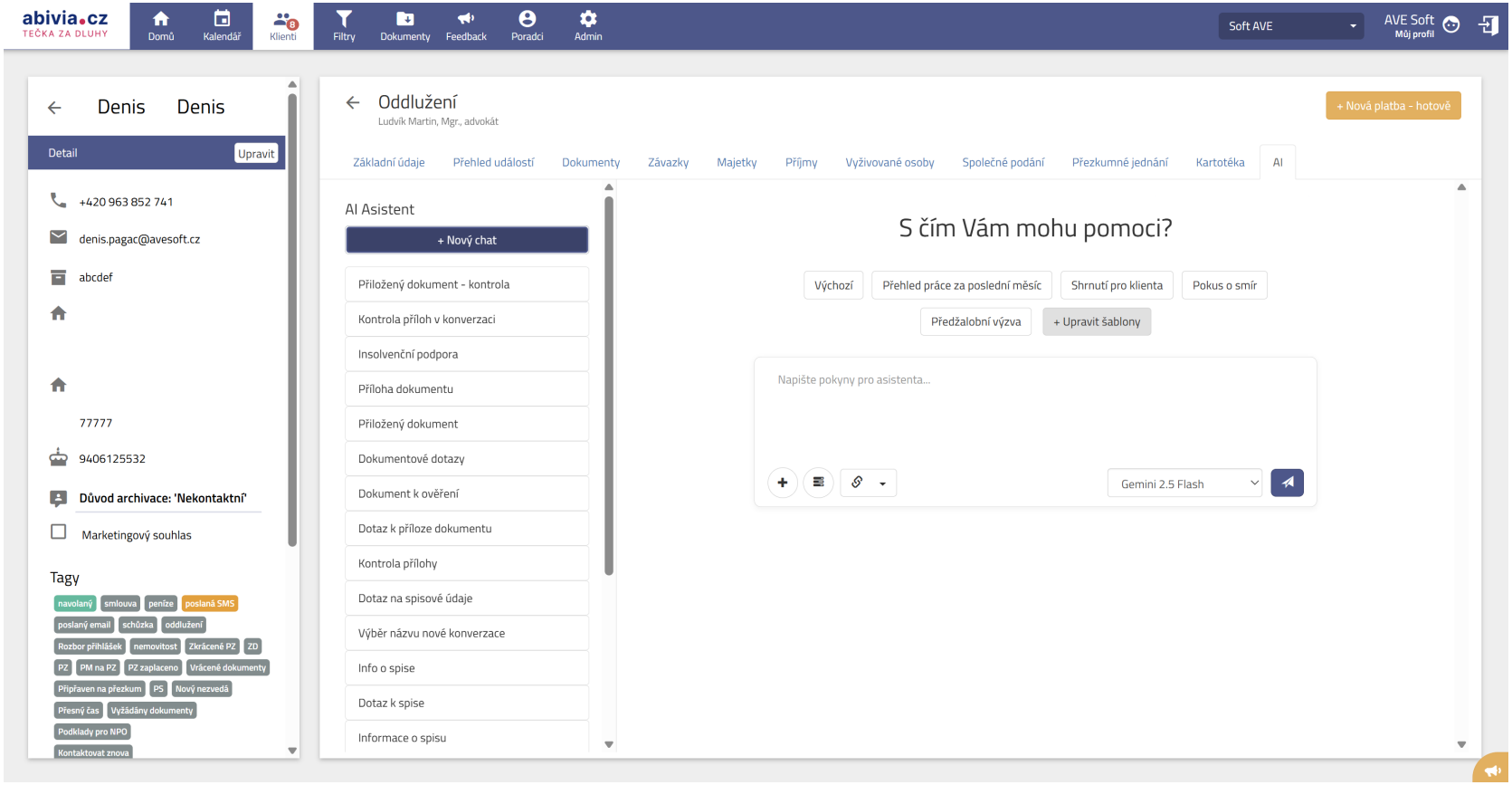
4.1 Implementace AI asistenta v prostředí AngularJS

Jedním z nejnáročnějších úkolů byl vývoj inteligentního chatu pro specifického klienta, jehož informační systém je postaven na starší technologii **AngularJS (v1.x)** [7]. Cílem bylo integrovat moderní jazykové modely (LLM) do tohoto prostředí a umožnit uživatelům nejen konverzaci, ale i analýzu nahraných dokumentů.

Je důležité podotknout, že framework AngularJS je v současnosti již ve fázi ukončené podpory (*end-of-life*, oficiálně od 31. 12. 2021 [8]) a v rámci standardních firemních procesů jej pro nové projekty již nenasazujeme. Volba této technologie byla v tomto případě čistě pragmatická a vynucená architekturou stávajícího informačního systému klienta. Cílem bylo vyhovět požadavku na integraci moderních funkcí do již zavedené aplikace a zajistit její další rozvoj bez nutnosti okamžité a nákladné migrace celého systému na modernější platformu.

4.1.1 Architektura řešení a datový tok

Modul je rozdělen do tří vrstev, které společně zajišťují celý životní cyklus konverzace – od odeslání dotazu až po streamování odpovědi zpět do uživatelského rozhraní. Jejich zodpovědnosti shrnuje tabulka 4.1.



Obrázek 4.1: Výchozí rozhraní AI asistenta s nabídkou předdefinovaných šablon (*prompt templates*) a volbou jazykového modelu

Tabulka 4.1: Vrstvy AI modulu a jejich zodpovědnosti

Vrstva	**Zodpovědnost**
Prezentační (AngularJS)	Správa vláken (*threads*) konverzace, rendering Markdown výstupu, upload dokumentů, sběr tokenů streamu.
Aplikační (C# Service Fabric)	Orchestrace: příjem dotazu, konstrukce promptu, volání LLM API, publikace tokenů do SignalR hubu.
Externí LLM API	Generování odpovědi (Azure OpenAI / OpenAI [9]); streaming tokenů přes rozhraní Chat Completions.

Vzhledem k požadavku na plynulé zobrazování odpovědi (podobně jako v chatovacích rozhraních typu ChatGPT) nebylo možné čekat na vygenerování celé odpovědi na serveru. Řešení využívá technologii **SignalR** [10] pro real-time přenos jednotlivých tokenů textu v momentě jejich vygenerování.

Datový tok od okamžiku odeslání dotazu je následující:

1. Frontend odešle HTTP POST požadavek s dotazem, identifikátorem vlákna (`partitionKey`, `rowKey`) a identifikátorem vybraného modelu (`selectedModelId`).

2. Backendová služba `AiAgents_Threads` dohledá konverzační vlákno v úložišti, načte jeho historii a sestaví kompletní prompt pro LLM (systémový prompt + historie zpráv + aktuální dotaz).

3. Backend iniciuje volání Chat Completions API s parametrem `stream = true`; přijaté SSE události postupně publikuje přes SignalR hub zpět na klienta.

4. Frontend je přihlášen k odběru události `openAIStream`; každý příchozí fragment (chunk) inkrementálně připojuje k aktuální zprávě a vynucuje překreslení UI.

5. Po ukončení streamu backend pošle řídicí zprávu `refreshStatus = REFRESH`, která klienta informuje o dokončení a zároveň uvolní blokaci odesílacího tlačítka.

```
// Naslouchání na notifikace ze SignalR hubu
hub.on('openAIStream', function(message) {
    $rootScope.$broadcast('openAIStream', message);
});

// Skládání zprávy v kontroleru (ev. zdroj: contractDetail.ts)
$scope.$on('openAIStream', function(event, chunk) {
    // Inkrementální připojení tokenu k aktuální streamované zprávě
```

```
    streamingMsg.Message += chunk;
    // Vynucení digest cyklu mimo Angular kontext
    $scope.$applyAsync();
});

// Ukončení streamu a uvolnění UI
$scope.$on('refreshStatus', function(event, status) {
    if (status === 'REFRESH') {
        $scope.isStreaming = false;
        $scope.isLoading = false;
    }
});
```

Listing 4.1: Zpracování streamované odpovědi v AngularJS

4.1.2 Správa vláken konverzace a historie

Každá konverzace je v systému reprezentována samostatným **vláknem** (*thread*) identifikovaným dvojicí `partitionKey` a `rowKey` (klíče úložiště Azure Table Storage v rámci Service Fabric služby). Vlákno uchovává:

- seznam všech zpráv (role `user` / `assistant`) v chronologickém pořadí,
- identifikátor zvoleného modelu (`idaiModel`), který je závazný pro celé vlákno,
- seznam vazeb na nahrané dokumenty (dokumenty se ukládají odděleně a do vlákna se připojují pouze reference).

Takto oddělené úložiště umožňuje efektivně iterovat nad historií bez nutnosti přenášet celé dokumenty, a zároveň poskytuje klientovi seznam vláken k výběru (boční panel konverzací).

Při odesílání nové zprávy frontend prostřednictvím metody `createThread()` buď pokračuje v existujícím vláknu, nebo jej nově zakládá. Výběr modelu je při prvním založení vlákna iniciálně nastaven na model označený v databázi jako výchozí (`jeVychozi = true`); uživatel jej může přepnout při vytváření vlákna, po zahájení konverzace je však model uzamčen pro zajištění konzistence odpovědí v rámci téhož vlákna.

4.1.3 Zpracování nahraných dokumentů

Klíčovým požadavkem byla analýza právních dokumentů. Tato funkce vyžadovala specifický dvoufázový proces:

4.1.3.0.1 Fáze 1 – Upload. Po výběru dokumentů uživatelem jsou soubory odeslány na endpoint `AiAgents_Docs_Upload_ById` voláním `$q.all()` pro paralelní zpracování. Backend každý dokument extrahuje do textové podoby (pro PDF, DOCX a obdobné formáty), uloží do trvalého úložiště a vrátí identifikátor `idDokument`. Během uploadu je UI blokováno (`uploadInProgress = true`) a odesílací tlačítko je deaktivováno.

4.1.3.0.2 Fáze 2 – Vazba na zprávu. Po dokončení uploadu všech souborů frontend přidá do aktuální zprávy kolekci referencí (`idDokument`, název souboru, přípona) a teprve poté zahájí samotné volání `createThread()`. Backend při sestavování promptu pro LLM načte extrahovaný obsah referencovaných dokumentů a zařadí jej do kontextového okna jazykového modelu. Tento přístup (označovaný jako *in-context document loading*) je jednodušší alternativou ke vzoru Retrieval-Augmented Generation (RAG) a je vhodný pro scénáře, kde je celý dokument dostatečně malý, aby se vešel do kontextového okna modelu.

4.1.4 Formátování výstupu a Markdown rendering

Jazykové modely generují odpověď ve formátu Markdown. Pro korektní zobrazení byla nasazena knihovna **marked.js**, která v reálném čase konvertuje text na HTML. Konfigurace potlačila automatické generování ID nadpisů a mangling e-mailových adres:

```
const markedInstance = window.marked;
if (markedInstance && typeof markedInstance.setOptions === 'function') {
    markedInstance.setOptions({ mangle: false, headerIds: false });
}
```

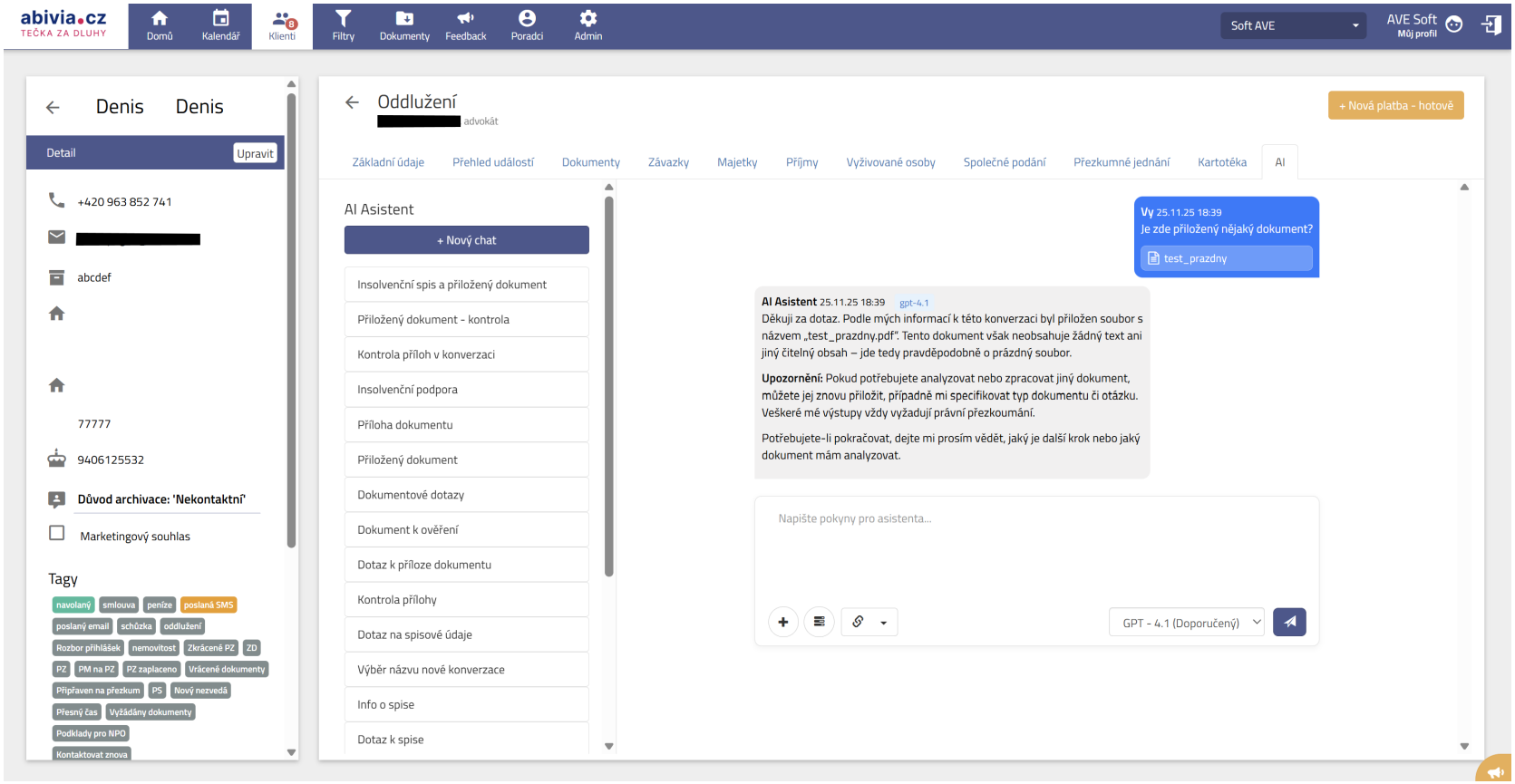
Listing 4.2: Konfigurace Markdown parseru (marked.js)

Protože je rendering volán při každém tokenu, bylo nutné ošetřit i částečně dokončené bloky (např. neukončený "‘ blok kódu); při nekonzistentním stavu zůstává text v surové podobě až do doručení dalšího fragmentu.

4.1.5 Ošetření chybových stavů

Streamované API přináší specifické chybové stavy, které klasické REST volání neřeší:

- **Přerušení streamu na backendu** – backend publikuje událost `openAIError`, klient uvolní UI a zobrazí informaci o selhání.
- **Selhání uploadu dokumentu** – při selhání kteréhokoliv souboru je `$q.all()` odmítnut, UI odstraní přidané zprávy a konverzace se vrátí do výchozího stavu.
- **Ztráta SignalR spojení** – klient volá `signalRService.connect(contextGuid)`, který při výpadku automaticky navazuje reconnect.



Obrázek 4.2: Ukázka probíhající konverzace s formátováním textu (Markdown) a analýzou nahraného dokumentu

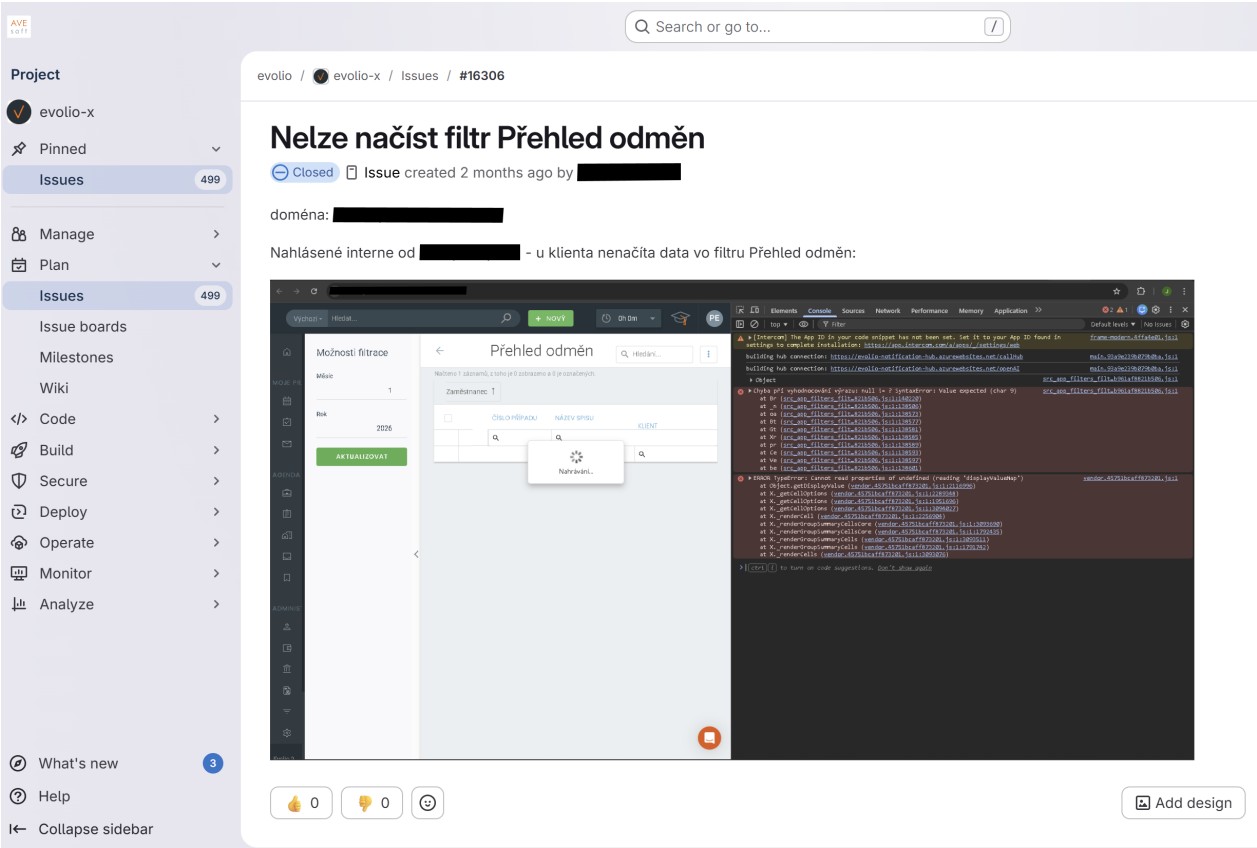
4.2 Řešení chyb v uživatelském rozhraní (opravy chyb)

Nedílnou součástí praxe byla analýza a opravy chyb v systému Evolio. Příkladem komplexního problému byla chyba v komponentě Přehledy (založené na knihovně DevExpress [11]), kde docházelo k selhání načítání filtrů.

4.2.1 Analýza problému: Konflikt seskupování a ukotvení

Uživatelé hlásili, že u určitých přehledů se nenačetla data v tabulce a došlo k permanentnímu stavu načítání (spinner se nezastavil). Analýzou bylo zjištěno, že problém nastává v momentě, kdy je sloupec nastaven jako **ukotvený** (Fixed) a zároveň se podle něj **seskupuje** (Grouped).

Komponenta DevExpress v této specifické konfiguraci nedokázala správně vypočítat pozici elementů, což vedlo k chybě v konzoli a k tomu, že se data tabulky nenačetla.



Obrázek 4.3: Ukázka zadaného požadavku (issue) v systému GitLab popisující nekonzistentní chování tabulky při kombinaci Fixed + Grouped

4.2.2 Implementace opravy

Oprava spočívala v úpravě logiky, která generuje konfiguraci pro sloupce tabulky. Bylo nutné zajistit, aby se vlastnost `fixed` automaticky deaktivovala, pokud je sloupec použit pro seskupování.

```
// Puvodni problematicky kod
// fixedPosition: colcf.fixed !== null ? colcf.fixed.toLowerCase() : null,
// fixed: colcf.fixed !== null,

// Nova logika s osetrenim konfliktu
const isFixed = colcf.fixed !== null &&
              colcf.fixed.toLowerCase() !== 'none' &&
              !colcf.grouped; // Klicova podminka: nesmi byt seskupeno

// Aplikace do konfigurace
const columnConfig = {
   // ... ostatni vlastnosti
   fixedPosition: isFixed ? colcf.fixed.toLowerCase() : null,
   fixed: isFixed
};
```

Listing 4.3: Úprava konfigurace sloupců pro detekci konfliktu Fixed/Grouped

Tato úprava zajistila stabilitu komponenty i v případě, že uživatel (nebo uložený přehled) definoval tuto konfliktní kombinaci vlastností.

Kapitola 5

Automatizované testování

Pro zajištění kvality a stability systému Evolio existuje v rámci firemní infrastruktury sada automatizovaných UI testů. Tyto testy simulují reálné chování uživatele v prohlížeči a slouží jako regresní testování po nasazení nové verze aplikace. V rámci praxe jsem se do tohoto projektu zapojil – rozšiřoval jsem stávající sadu o nové testovací scénáře a průběžně pečoval o funkčnost již existujících testů.

5.1 Metodika a zařazení v testovací pyramidě

Automatizované testování softwaru bývá zpravidla strukturováno podle tzv. *testovací pyramidy* [12, 13], která rozlišuje tři vrstvy testů podle granularity a rychlosti zpětné vazby:

Tabulka 5.1: Vrstvy testovací pyramidy a jejich pokrytí v projektu Evolio

Vrstva	**Účel**	**Nástroje v Evoliu**	**Moje zapojení**
Unit testy	Izolované testování jednotek kódu (metody, třídy).	NUnit, Moq (v dílčích modulech).	Minimální – moduly, na kterých jsem pracoval, mají nízké pokrytí unit testy.
Integrační testy	Ověření spolupráce více komponent (služba + DB).	Vlastní framework nad .NET + testovací SQL instance.	Žádné – nebyly předmětem praxe.
UI / E2E testy	Simulace uživatele v prohlížeči; validace end-to-end scénářů.	Selenium WebDriver, NUnit, Boa.Constrictor.	Hlavní oblast praxe – rozšiřování a údržba.

UI testy stojí na vrcholu pyramidy; jsou nejpomalejší a nejnáchylnější k nestabilitě (tzv. *flaky tests*), ale zároveň poskytují nejbližší aproximaci reálného uživatelského scénáře. Jejich hodnota

spočívá v tom, že odhalí chyby vznikající na rozhraní mezi vrstvami – například selhání komunikace mezi frontendem a backendem nebo chybně vyrenderované UI prvky, které unit test nemůže odhalit.

5.2 Strategie a nástroje

Jako technologický základ byl zvolen framework **Selenium WebDriver** [14], který je v .NET prostředí de facto standardem pro automatizaci webových prohlížečů. Pro samotné spouštění testů a validaci podmínek (assertions) je využit framework **NUnit** [15] (atributy `[Test]`, `[SetUp]`, `[TearDown]`).

5.2.1 Volba návrhového vzoru: Screenplay místo Page Object Modelu

Pro organizaci testovacího kódu existují dva převládající návrhové vzory. Jejich srovnání shrnuje tabulka 5.2.

Tabulka 5.2: Porovnání Page Object Model a Screenplay Pattern

Aspekt	**Page Object Model (POM)**	**Screenplay Pattern**
Hlavní abstrakce	Stránka (třída reprezentující jednu obrazovku).	Herec (Actor) a jeho akce.
Organizace kódu	Metody jsou vázány na stránku.	Akce jsou samostatné, znovupoužitelné.
Čitelnost testu	Procedurální styl.	Deklarativní, čitelný jako přirozený jazyk.
Škálovatelnost	Při růstu se objevují „God Object"třídy.	Lepší kompozice; nové akce bez zásahu do stránek.
Křivka učení	Nízká.	Vyšší – vyžaduje pochopení SOLID principů.

Projekt UI testů v Evoliu využívá **Screenplay Pattern** [16] implementovaný pomocí knihovny **Boa.Constrictor** [17]. Tento přístup se zaměřuje na to, *co* uživatel (Actor) dělá, spíše než na strukturu stránek, a odpovídá principům SOLID – zejména Single Responsibility Principle, neboť každá akce má jediný důvod ke změně [18].

Základní stavební kameny architektury jsou:

- **Actor (Tester):** Entita provádějící test. V našem případě je instanciována v `BaseTest` třídě.
- **Task:** Složitější úloha skládající se z více kroků (např. `Login.WithSessionCredentials()`).
- **Interaction:** Atomická akce (např. `Click.On(...)`, `SendKeys.To(...)`).

- **Question:** Dotaz na stav aplikace (např. `Appearance.Of(...)`, `ValueAttribute.Of(...)`).

5.3 Implementace testovacích scénářů

Testy jsou organizovány do logických celků podle agend (Podatelna, Případy, Subjekty). Následující ukázka demonstruje strukturu CRUD testu pro vytvoření nového subjektu.

5.3.1 Příklad: Vytvoření subjektu

Testovací metoda dodržuje princip **Arrange-Act-Assert** [19]. Využívá fluent syntaxi, která činí test čitelným téměř jako přirozený jazyk.

```
[Test]
public void CreateSubject_Manually_ShouldSucceed()
{
    // Arrange: Priprava dat
    string name = "Testovaci Subjekt " + Guid.NewGuid();
    string ico = "12345678";

    // Act: Provedeni akci (Navigace -> Vyplneni formulare -> Ulozeni)
    Tester.AttemptsTo(Navigate.ToUrl(SubjectPage.Url));
    Tester.AttemptsTo(Click.On(SubjectPage.AddButton));

    Tester.AttemptsTo(SendKeys.To(SubjectForm.NameInput, name));
    Tester.AttemptsTo(SendKeys.To(SubjectForm.IcoInput, ico));

    Tester.AttemptsTo(Click.On(SubjectForm.SaveButton));

    // Assert: Overeni vysledku
    // Cekame, az se objevi notifikace o uspechu
    Tester.WaitsUntil(Appearance.Of(SubjectPage.SuccessToast), IsEqualTo.True());

    // Overeni, ze se subjekt nacetl v detailu
    Tester.AskingFor(Text.Of(SubjectPage.DetailHeader)).Should().Contain(name);
}
```

Listing 5.1: Ukázka testu vytvoření subjektu pomocí Screenplay patternu

5.3.2 Správa testovacích dat a izolace

Zajištění opakovatelnosti testů je v UI prostředí kritické, neboť chyby v datové izolaci vedou k nejčastější příčině nestabilních testů. V projektu jsou uplatňovány tyto principy:

- **Unikátní identifikátory** – každý vytvářený záznam dostává na vstupu `Guid.NewGuid()` v názvu, takže nedojde ke kolizi s artefakty z předchozích běhů.
- **Izolovaná databáze** – testy se spouštějí proti samostatné testovací instanci SQL Serveru, která je resetována před každým release buildem.
- **Session credentials** – přihlášení je sdíleno napříč testy v rámci jedné třídy přes atribut `[OneTimeSetUp]`, což zkracuje celkový běh.

5.3.3 Ošetření nestability (flakiness)

Explicitní čekání pomocí `Tester.WaitsUntil(...)` eliminuje významnou část nestability způsobenou asynchronním načítáním dat. Tam, kde je chování časově nedeterministické (např. závislost na AI API nebo na externí službě), je použit interní retry mechanismus knihovny Boa.Constrictor, který akci opakuje s definovaným intervalem a maximálním počtem pokusů.

5.4 Integrace do CI/CD (Azure DevOps)

Testovací projekt je integrován do procesu kontinuální integrace a nasazování (CI/CD) na platformě **Azure DevOps** [20], kde jsem se podílel na správě buildovacích a release pipeline.

5.4.1 Spouštění testů

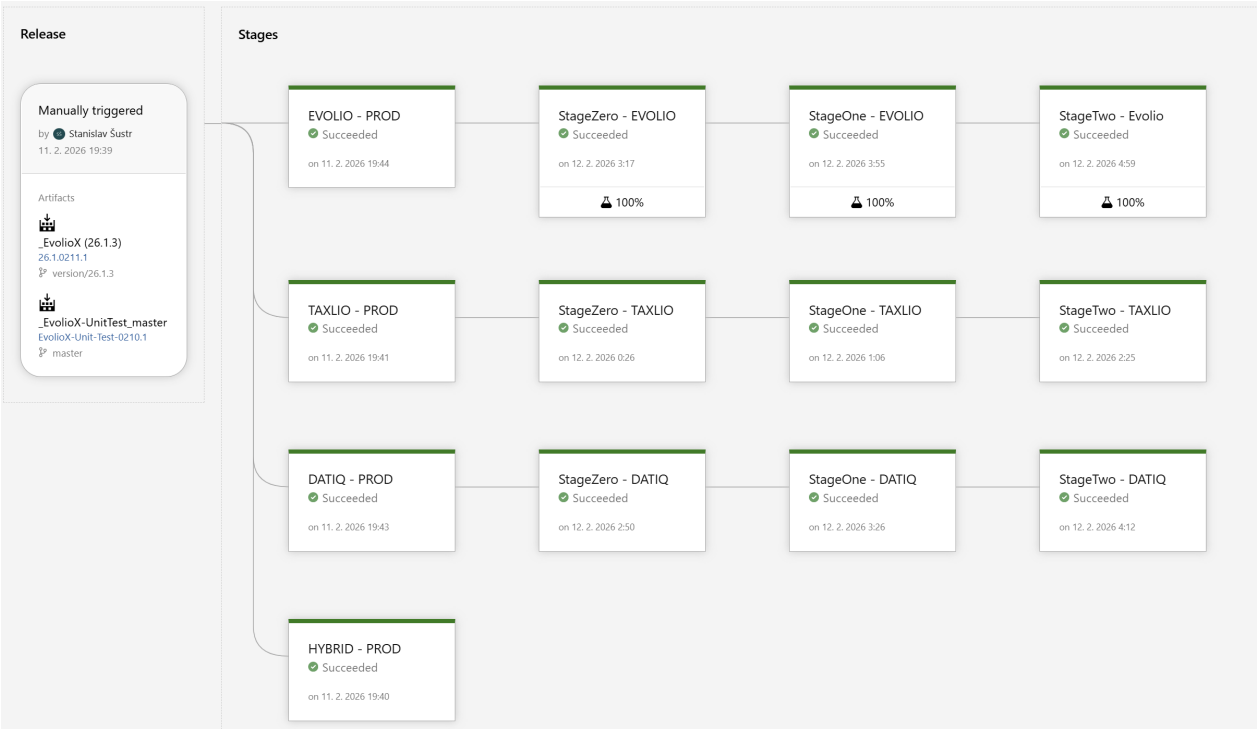
Testy jsou automaticky spouštěny po každém nasazení nové verze do testovacího prostředí. Běží na vzdáleném virtuálním stroji v prohlížeči Google Chrome v režimu headless, což zkracuje dobu běhu a umožňuje paralelní spouštění více testovacích běhů.

5.4.2 Diagnostika chyb

Pro efektivní řešení problémů je implementován mechanismus, který v případě pádu testu:

1. Automaticky pořídí **screenshot** obrazovky v momentě chyby.
2. Uloží systémové logy a stack trace.
3. Tyto artefakty připojí k výsledku testu v Azure DevOps Dashboardu.

Tato automatizace výrazně zrychluje proces debugování, protože umožňuje okamžitě identifikovat, zda chyba nastala v aplikaci (např. validační hláška) nebo v testu samotném.



Obrázek 5.1: Ukázka úspěšného běhu automatizovaných testů v release pipeline Azure DevOps

5.5 Přínos testovací sady

Sada automatizovaných testů již v průběhu mé praxe opakovaně odhalila regresní chyby ještě před nasazením k zákazníkům – konkrétní příklad je uveden v kapitole 8. Současný stav pokrývá klíčové CRUD operace hlavních agend (Podatelna, Případy, Subjekty, Fakturace) a jeho rozšíření o nové scénáře bylo jednou z mých průběžných činností. Pro další růst kvality by bylo vhodné doplnit nižší vrstvy pyramidy – zejména integrační testy serverové logiky Add-inů, kde jsou dnes regrese odhalovány až UI testy, což je nákladnější z hlediska doby běhu a diagnostiky.

Kapitola 6

Vazba praxe na studium

Odborná praxe mi umožnila konfrontovat teoretické znalosti získané studiem na Fakultě elektrotechniky a informatiky VŠB-TUO s reálnými požadavky softwarového vývoje. V mnoha oblastech jsem mohl přímo navázat na absolvované předměty, zatímco jiné domény vyžadovaly intenzivní samostudium. Tato kapitola se zaměřuje na konkrétní využití školní látky v praxi; přehled nově osvojených technologií a chybějících znalostí je věnována samostatná kapitola 7.

6.1 Uplatnění teoretických znalostí

Nejvýznamnějším přínosem studia pro tuto praxi byla znalost objektově orientovaného programování a databázových systémů. Konkrétně jsem využil znalosti z následujících předmětů:

- **Programování v C# 1 a 2:** Znalost syntaxe jazyka C#, typového systému a asynchronního programování byla nezbytná pro vývoj backendové části (Add-in) a psaní automatizovaných testů. Konkrétním příkladem bylo využití `async/await` a generických typů při implementaci metody `DirectImportAsync`, kde bylo nutné správně zpracovat asynchronní volání interního API systému. Bez pevného základu z těchto předmětů bych nebyl schopen rychle pochopit strukturu existujícího produkčního kódu a do něj přispívat.

- **Databázové systémy 1 a 2:** Principy návrhu databází jsem využil při spolupráci na návrhu schématu pro modul Importér. Pomohlo mi, že jsem rozuměl tomu, proč je důležité správně definovat vztahy mezi tabulkami a zajistit integritu dat již na úrovni databáze. Stejně tak mi znalost SQL usnadnila psaní uložených procedur a orientaci v existujících dotazech systému Evolio.

- **Funkcionální programování (FPR):** Ačkoliv je většina projektu psána v imperativních jazycích, principy rekurze probírané v tomto předmětu byly klíčové při implementaci hierarchického stromu kontextů (*Context Tree*). Zobrazení vnořených struktur (např. Subjekt →

Adresa → Kontakt) vyžadovalo rekurzivní průchod daty jak v databázovém schématu (rekurzivní vazba `IdKontextNadrizeny`), tak i ve frontendové komponentě `ContextTreeComponent`, která se sama volá pro každou úroveň stromu. Tento koncept jsem si pevně osvojil právě díky výuce FPR.

- **Správa softwarových projektů (SSP):** Základy verzování kódu pomocí systému Git, představené v tomto předmětu, jsem využíval denně při práci s větvemi (*branches*) a správě zdrojových kódů. Firemní procesní nadstavbu tohoto tématu (code review, konvence commitů) rozebírá kapitola 7.

6.2 Přechod od teorie k praxi

Přestože výše zmíněné předměty poskytly solidní základ, praxe odhalila, že rozdíl mezi akademickým a průmyslovým pojetím programování není jen v technologiích, ale především v měřítku a kontextu. Školní projekty jsou zpravidla budovány od nuly, mají jasně ohraničený rozsah a nejsou svázány existující architekturou. V praxi naopak probíhá vývoj v rámci rozsáhlého systému s mnohaletou historií, kde každá změna musí respektovat zavedené konvence, nenarušit existující funkčnost a projít revizí kolegů.

Tato zkušenost pro mě byla klíčová: potvrdila, že teoretický základ ze školy je nezbytnou podmínkou, nikoli však postačující. Schopnost číst a pochopit cizí produkční kód, orientovat se v architektuře systému, který člověk sám nevytvořil, a komunikovat technická rozhodnutí v týmu – to jsou dovednosti, které nelze plně osvojit v akademickém prostředí a které jsem si mohl vybudovat právě až v průběhu praxe.

Kapitola 7

Rozvoj kompetencí a chybějící znalosti

Přechod do firemního prostředí odhalil rozdíly mezi akademickými projekty a vývojem komerčního softwaru. Tato kapitola shrnuje technologie, které jsem se musel naučit samostudiem v průběhu praxe, a procesní aspekty, které ve výuce chyběly nebo byly pokryty jen povrchně.

7.1 Nově osvojené technologie

Největší výzvou byl vývoj moderního frontendu. Ačkoliv předmět *Tvorba aplikací pro mobilní zařízení I* poskytuje základy jazyka JavaScript (Vanilla JS), pro práci na projektu Importér bylo nutné ovládnout komplexní ekosystém frameworku Angular a jazyka TypeScript.

7.1.1 Reaktivní programování s RxJS

Největším technologickým skokem bylo pochopení reaktivního programování. Koncept `Observable` — datového toku, na který se přihlásíme k odběru a reagujeme na příchozí hodnoty — byl zpočátku velmi odlišný od imperativního stylu, na který jsem byl zvyklý ze školních projektů.

V praxi jsem se nejčastěji setkával s operátory jako `forkJoin` pro paralelní volání více API najednou, `switchMap` pro zrušení předchozího požadavku při rychlém přepisování vstupu a `takeUntil` pro bezpečné odhlášení od odběru při zničení komponenty. Bez jejich pochopení by kód byl buď nefunkční, nebo by docházelo k únikům paměti (tzv. *memory leaks*).

7.1.2 Angular architektura

Angular jako framework přináší přísnou strukturu, která zpočátku působí jako zbytečná komplexita, ale při vývoji rozsáhlejšího modulu se ukázala jako velká výhoda. Musel jsem pochopit systém Dependency Injection, který řídí, jak si komponenty a služby předávají závislosti, životní cyklus komponent (hooks jako `ngOnInit`, `ngOnDestroy`) a způsob, jakým Angular detekuje změny v datech a kdy překresluje UI.

Zvláštní pozornost si vyžádala strategie **OnPush**, kterou jsem musel nasadit u všech náročnějších komponent modulu Importér kvůli výkonu. Pochopení toho, kdy Angular komponentu překreslí a kdy ne, bylo klíčové pro odladění chyb, kde se UI neaktualizovalo po změně dat.

7.1.3 Azure Service Fabric a architektura mikroslužeb

Systém Evolio je postaven na platformě Azure Service Fabric, kde každá funkcionalita existuje jako samostatná služba (Actor nebo Service) s vlastním životním cyklem. Pro mě jako studenta zvyklého na monolitické aplikace bylo zpočátku obtížné pochopit, jak spolu tyto služby komunikují, jak se ladí chyba, která přechází přes hranice více služeb, a jak správně nastavit lokální vývojové prostředí tak, aby všechny závislosti fungovaly.

7.2 Chybějící procesní znalosti

Vedle technologií jsem si musel osvojit i řadu procesních dovedností, které jsou ve firemním prostředí samozřejmostí, ale ve školní výuce nejsou systematicky pokryty.

7.2.1 Code Review a kultura zpětné vazby

Každý kód, který jsem napsal, procházel před začleněním do hlavní větve revizí kolegy. Zpočátku bylo náročné přijímat připomínky k vlastnímu kódu a chápat je jako přirozenou součást procesu, nikoliv jako kritiku. Postupně jsem si ale zvykl na to, že code review odhaluje věci, které autor sám přehlédne, a smysl tohoto přístupu jsem ocenil velmi rychle – naučil jsem se z komentářů kolegů aktivně čerpat znalosti o firemních konvencích a architektonických rozhodnutích.

7.2.2 Práce v rozsáhlé sdílené kódové základně

Na rozdíl od školních projektů, kde celý kód píše jeden člověk nebo malá skupina, systém Evolio tvoří stovky souborů vyvíjených souběžně více vývojáři. Naučil jsem se orientovat v cizím kódu, dohledávat kontext změn přes historii Gitu a dbát na to, aby moje změny nezasáhly do oblastí, na kterých paralelně pracuje kolega — čemuž slouží právě striktní větvení a pravidelné mergování.

Kapitola 8

Zhodnocení výsledků a přínos praxe

Cílem praxe bylo nejen zapojení do běžného chodu firmy, ale především dodání konkrétních softwarových řešení. Následující sekce uvádějí aktuální stav jednotlivých projektů, dosažené výstupy a kritickou reflexi vlastní práce.

8.1 Modul Importér

V době psaní této práce se nová webová verze Importéru nachází ve fázi **intenzivního testování**. Aplikace je plně funkční a probíhají konzultace s vedoucím týmu (*team leader*) a produktovým manažerem (*product owner*) ohledně doladění vizuálních detailů a uživatelského prožitku (UX).

Do testování jsou zapojeni také firemní konzultanti, kteří budou systém v budoucnu nejvíce využívat. Jejich zpětná vazba slouží k identifikaci posledních nedostatků před nasazením do ostrého provozu pro všechny klienty.

8.1.1 Rozsah realizace

V rámci modulu bylo vytvořeno:

- osm specializovaných Angular komponent (viz tabulka 3.2),
- jeden samostatný feature modul (`ImporterModule`) s vlastním routingem a *lazy loadingem*,
- dva C# Add-iny zajišťující dvoufázové zpracování importu,
- devět nových databázových tabulek pod schématem `importer` a několik podpůrných uložených procedur.

8.1.2 Přínos pro firmu

Po nasazení nového modulu se očekává výrazné snížení zátěže firemních konzultantů, kteří dosud museli importy provádět za klienty, a zpřístupnění této funkce širšímu okruhu uživatelů.

8.2 AI rozhraní

Modul AI chatu byl dokončen, nasazen do produkčního prostředí a je klientem aktivně využíván, přičemž klient vyjádřil s výsledkem spokojenost. Implementace prokázala, že i zavedený informační systém postavený na starší technologii (AngularJS) lze funkčně obohatit o moderní funkce bez nutnosti kompletní migrace platformy.

8.3 Automatizované testy

Sada automatizovaných testů byla integrována do CI/CD pipeliny a spouští se s každým vydáním nové verze. Testy již prokázaly svou reálnou hodnotu při odhalování regresních chyb.

Konkrétním příkladem úspěšného záchytu chyby byla situace, kdy testy odhalily nefunkčnost modálního okna pro editaci případu, pokud neobsahoval žádné subjekty. Chyba způsobovala permanentní stav načítání a zablokování formuláře. Díky automatizaci byla tato kritická chyba zachycena ještě před nasazením k zákazníkům.

8.4 Kritická sebereflexe

Ne všechno se podařilo tak, jak jsem si původně představoval. K nejvýznamnějším oblastem, kde vidím prostor pro osobní zlepšení, patří:

- **Odhad časové náročnosti.** První úkoly v modulu Importér trvaly výrazně déle, než jsem odhadoval – především kvůli podcenění doby potřebné na pochopení existujícího kódu a firemních konvencí. V pozdější fázi praxe se přesnost mých odhadů znatelně zlepšila.
- **Pokrytí nižšími vrstvami testů.** Moje pozornost směřovala primárně k UI testům, ačkoliv by pro některé komplexní Add-in scénáře byly integrační testy efektivnější (rychlejší zpětná vazba, jednodušší diagnostika). Toto je oblast, na kterou bych v dalším období rád navázal.
- **Dokumentace vlastního kódu.** Zpětně vnímám, že některé mé první implementace by zasloužily podrobnější komentáře a dokumentační bloky. V průběhu praxe jsem tento návyk postupně zapracoval.

8.5 Budoucí rozvoj

V rámci Importéru plánuji ve firmě pokračovat v těchto směrech:

- rozšíření o další entity (dokumenty, finance),
- doplnění integračních testů Add-in vrstvy,
- AI-asistovaný návrh mapování podle hlaviček souboru.

Kapitola 9

Závěr

Cílem mé odborné praxe ve společnosti AVE soft s.r.o. bylo zapojení se do reálného vývojového procesu a realizace konkrétních softwarových úkolů v rámci informačního systému Evolio. Hlavním výstupem byl návrh a implementace webového rozhraní pro modul Importér, který nahrazuje zastaralé desktopové řešení postavené na technologii WPF.

Během praxe se podařilo vytvořit moderní Single Page Aplikaci v technologii Angular, která komunikuje s nově vytvořeným backendovým doplňkem v jazyce C#. Součástí řešení je také návrh databázového schématu postaveného s důrazem na konfigurovatelnost a auditovatelnost, který umožňuje definovat nové typy importů bez zásahů do zkompilovaného kódu. Ačkoliv je modul momentálně ve fázi finalizace a testování, již nyní nabízí intuitivnější uživatelské rozhraní a zjednodušuje proces importu dat natolik, že jej budou moci provádět samotní klienti.

Vedlejší úkoly – implementace AI asistenta pro právního klienta a rozšíření sady automatizovaných UI testů – byly dokončeny a nasazeny do produkčního prostředí, přičemž testovací sada již v průběhu praxe prokazatelně zachytila několik regresních chyb před jejich únikem k zákazníkům.

Z hlediska osobního rozvoje byla praxe přínosem, který akademické studium nemůže plně nahradit. Osvojil jsem si moderní technologie, které v průběhu studia nebyly vyučovány (Angular, RxJS, Signals, SignalR, Screenplay Pattern), ale zejména jsem získal zkušenost s vývojem v rozsáhlé sdílené kódové základně, s procesem code review, s prací v Agile týmu a s CI/CD nástroji na platformě Azure DevOps. Tato praxe mi potvrdila, že kvalitní teoretický základ ze školy je nezbytným předpokladem profesního uplatnění, ale sám o sobě nestačí – dlouhodobá konkurenceschopnost v oboru je podmíněna ochotou neustále se učit, pracovat v týmu a přijímat zpětnou vazbu.

Ve spolupráci se společností AVE soft s.r.o. budu pokračovat i po ukončení praxe, a to jak na dokončení modulu Importér, tak na dalších projektech, které mi firma nabídla.

Literatura

1. MICROSOFT CORPORATION. *.NET documentation* [online]. 2025. [cit. 2026-01-22]. Dostupné z: https://learn.microsoft.com/en-us/dotnet/.
2. GOOGLE LLC. *Angular - The web development framework for building the future* [online]. 2025. [cit. 2026-02-14]. Dostupné z: https://angular.dev/overview.
3. MICROSOFT CORPORATION. *TypeScript: JavaScript With Syntax For Types* [online]. 2025. [cit. 2026-02-14]. Dostupné z: https://www.typescriptlang.org/docs/.
4. MICROSOFT CORPORATION. *C# documentation* [online]. 2025. [cit. 2026-01-22]. Dostupné z: https://learn.microsoft.com/en-us/dotnet/csharp/.
5. MICROSOFT CORPORATION. *SQL Server technical documentation* [online]. 2025. [cit. 2026-01-30]. Dostupné z: https://learn.microsoft.com/en-us/sql/sql-server/.
6. RXJS CORE TEAM. *RxJS: Reactive Extensions Library for JavaScript* [online]. 2024. [cit. 2026-02-14]. Dostupné z: https://rxjs.dev/guide/overview.
7. GOOGLE LLC. *AngularJS developer guide* [online]. 2021. [cit. 2026-02-17]. Dostupné z: https://docs.angularjs.org/guide.
8. GOOGLE ANGULAR TEAM. *Finding a path forward with AngularJS* [online]. 2021. [cit. 2026-02-17]. Dostupné z: https://blog.angular.dev/finding-a-path-forward-with-angularjs-7e186fdd4429.
9. OPENAI. *OpenAI API Reference – Chat Completions and Streaming* [online]. 2025. [cit. 2026-02-17]. Dostupné z: https://platform.openai.com/docs/api-reference/chat.
10. MICROSOFT CORPORATION. *Introduction to ASP.NET Core SignalR* [online]. 2024. [cit. 2026-02-17]. Dostupné z: https://learn.microsoft.com/en-us/aspnet/core/signalr/introduction.
11. DEVELOPER EXPRESS INC. *DevExpress UI Components documentation* [online]. 2025. [cit. 2026-02-05]. Dostupné z: https://docs.devexpress.com/.
12. VOCKE, Ham. *The Practical Test Pyramid* [online]. 2018. [cit. 2026-02-10]. Dostupné z: https://martinfowler.com/articles/practical-test-pyramid.html.

13. MESZAROS, Gerard. *xUnit Test Patterns: Refactoring Test Code*. Addison-Wesley Professional, 2007. ISBN 978-0131495050.

14. SOFTWARE FREEDOM CONSERVANCY. *Selenium WebDriver documentation* [online]. 2025. [cit. 2026-02-10]. Dostupné z: `https://www.selenium.dev/documentation/webdriver/`.

15. NUNIT.ORG. *NUnit documentation* [online]. 2024. [cit. 2026-02-10]. Dostupné z: `https://docs.nunit.org/`.

16. KNIGHT, Andrew. *Screenplay Pattern: An Introduction* [online]. 2020. [cit. 2026-02-10]. Dostupné z: `https://automationpanda.com/2020/05/04/screenplay-pattern-an-introduction/`.

17. PALO ALTO NETWORKS. *Boa.Constrictor: The .NET Screenplay Pattern* [online]. 2024. [cit. 2026-02-03]. Dostupné z: `https://github.com/q2ebanking/boa-constrictor`.

18. MARTIN, Robert C. *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall, 2017. ISBN 978-0134494166.

19. BECK, Kent. *Test Driven Development: By Example*. Addison-Wesley Professional, 2002. ISBN 978-0321146533.

20. MICROSOFT CORPORATION. *Azure DevOps documentation* [online]. 2025. [cit. 2026-02-20]. Dostupné z: `https://learn.microsoft.com/en-us/azure/devops/`.